

0713 update — gate v2 端到端检索结果 + beam search 新思路

承接 [0705update.md](#)：那份文档记录了 per-j 混层 gate([gate/artifacts_pooled_v2](#)) 在 gate 自身指标(TPR/FPR/F1)上的全量验证结论，但当时明确留了一条"下一步"没做——端到端接回检索流程，看这个提升能不能传导到真实检索/答案指标。这次把这个补上了，同时记录两个还在探索阶段的新思路（beam search、以及把拆解也纳入 beam search），跟"继续优化 gate/decompose 单个模块"是不同性质的方向。

一、gate v2 端到端检索结果（已完成的验证）

用 `run_retrieval_exp_wavefront.py --methods gated_rule_a, --artifacts-dir` 分别指向生产 [gate/artifacts_pooled](#)（31 层，基线）和 [gate/artifacts_pooled_v2](#)（per-j 混层），在 `musique / 2wiki / hotpot` 三个数据集的 dev 全量上跑了一遍，逐字段对比：

数据集	指标	baseline(31层)	gate_v2(混层)	差值
MuSiQue	recall@1	0.6513	0.6565	+0.0052
	chain_recall@1	0.4233	0.4290	+0.0057
	answer EM / F1	0.4079 / 0.5012	0.4042 / 0.4983	-0.0037 / -0.0029
2Wiki	recall@1	0.8484	0.8569	+0.0085
	chain_recall@1	0.6684	0.6819	+0.0135
	answer EM / F1	0.4753 / 0.5493	0.4758 / 0.5499	+0.0005 / +0.0006
HotpotQA	recall@1	0.6186	0.6257	+0.0071
	chain_recall@1	0.4093	0.4112	+0.0019
	answer EM / F1	0.5130 / 0.6396	0.5129 / 0.6397	-0.0001 / +0.0001

结论：

- 1. 检索质量指标（**recall@1/@3、MRR、chain_recall**）在三个数据集上一致提升——这是比 0705update.md 里"gate 自身 TPR/FPR/F1 变好"更有说服力的证据：三个数据集构造方式、领域都不一样，跨数据集一致 变好，不太像是训练集分布内的巧合。
- 2. 但最终答案 **EM/F1** 几乎没有变化（三个数据集都在 $\pm 0.001 \sim 0.004$ 内浮动，musique 上甚至略微下降）。检索变好了，没有传导成下游任务的可衡量收益，说明当前流水线里 reader 这一环可能才是瓶颈，而不是检索/gate 这一段。
- 3. gate 触发率的变化方向在数据集间不一致：musique 上更少触发（0.3251→0.2958，更谨慎），2wiki/hotpot 上更多触发（0.1748→0.2167，0.2515→0.2789）——不是一个统一的"更保守"或"更激进"。

要不要转正：目前看是"检索层面确实更好，但对最终任务没有可衡量的收益"这个结论，转不转正需要 权衡"检索指标好看"和"没有下游收益"这两件事，属于人为决策，这份文档只提供数据支撑。

产出物：[retrieval/results/20260628_120339_wavefront_pooled_lam0p25](#) (baseline)、[retrieval/results/20260706_153235_gate_v2_single_gpu](#) (gate v2)。

二、为什么要找新思路，而不是继续优化 gate / decompose

上面这个结果暗示：单独在 gate 或 decompose 层面做完善式的改动，边际收益可能已经不大了一—gate 这次 已经是"per-j 混层"这种级别的改动，检索指标涨了，答案指标却没跟着涨；继续在这个模块里调参，大概率 也就是修修补补，很难指望有质变。所以这次往下想的两个方向，**都不是"改进某个模块的效果"，而是"这个模块产出的隐状态信号能不能用出新的方式"**——跟一直以来"用 hidden state 判断推理链是否正常" 这个核心想法一致，只是用法不一样。

三、新思路 1：Beam Search（已实现，待真实 GPU 验证）

动机

现在的检索流程每一跳贪心地选一个候选证据就往下走，只在选择那一刻用 gate 判断"这一步像不像错的"。如果某一跳选错了，后面的跳只能将错就错。想法是：每一跳保留多条候选路径 (beam)，用隐状态给每条路径打分，分数差的路径随时剪掉——这样即使某一跳的错误在当时看不出来，只要它在后续跳的隐状态里 留下了痕迹，还有机会在后面被淘汰。

设计

- **路径打分 = 累计异常分之和**，不掺相似度。相似度只用在"每条路径自己检索候选"这一步（先筛出 值得打分的候选，不然要给整个候选池都打分，太浪费）；跨路径比较、决定留哪几条，只看 gate 异常分。
- **没有"选 1 of 3"的 LLM 调用**——beam 剪枝本身就是选择机制，比贪心版本反而少一步。
- **每一跳流程**：beam 里 B 条路径各自展开子问题（可能因为前面选的证据不同，中间答案不同，展开出来的子问题也不同）→ 各自检索 top-k 候选 → 打包成 B×k 个"路径+候选"的延续，一次性批量过 gate 打分 → 按新的累计分排序，全局剪枝回 B 条 → 只对剪枝后活下来的 B 条生成中间答案（不是 B×k 条，省了不少 vLLM 调用）。
- **最终答案**：不把多条 beam 的证据链都塞进一个 prompt 给模型判断（怕上下文太长、模型读串），直接 用 beam 内部已经算出来的累计异常分最低的那一条，走现有单链最终答案生成流程，改动最小。

实现状态

新脚本 [error_propagation_probe/beam_search_retrieval.py](#)，大量 import 复用 [retrieval/run_retrieval_exp.py](#) 和 [run_retrieval_exp_wavefront.py](#) 里现成的数据加载 / 检索 / 批量 gate 打分 ([score_gate_requests](#)) / vLLM 生成封装，两个生产脚本一行没改。用假的 **gate 打分 + 假的 vLLM 生成、但真实的数据集加载和检索**跑通了完整流程：beam 从 1 条自然长到 3 条、多跳剪枝、只对幸存路径生成答案、最终选分数最低的 beam 生成答案、写出 JSONL——结构验证通过，真实的检索/答案 质量数字还没跑（需要真实 GPU + Llama + vLLM）。

真实跑法：

```
cd error_propagation_probe
python beam_search_retrieval.py --limit 20 --beam-width 3
```

四、新思路 2：把拆解 (decompose) 也纳入同一个 beam (设计已讨论, 未实现)

动机

decompose 现在是独立预处理步骤, BART 只输出 top-1 拆解, 好不好完全由 BART 自己训练时的目标 (BLEU/exact-match 对 GT 子问题) 决定, 跟"这个拆解到不到最后有没有帮助检索/推理"没有直接关系。既然核心想法是"用隐状态判断推理链正不正常", 那这把尺子也应该能用来评判"拆解选得好不好", 而不是让拆解继续用一套完全独立的、跟下游脱节的指标去优化。

设计 (两种方案, 取舍不同)

方案 A (成本几乎为零, 优先尝试): 不训练任何新东西, 把"选拆解"这件事折进现有的 evidence beam search 里, 让它被现有 gate 间接评判。

动手写这部分代码之前, 必须先验证一个前提, 否则整个方案 A 可能没有意义: BART 现在 `num_beams=10` 解码时, 排名第 2、3...名的候选, 到底是不是跟第 1 名有实质差异的、多样化的拆解, 还是几乎一样、只差一两个词的近似重复? 如果候选之间高度雷同, 那"多留几个候选给 beam 去选"根本没有东西可选; 另外还要看, 扩大到 **top-3/top-5** 范围之后, 正确 (或更接近 GT) 的拆解到底会不会出现——如果正确答案基本只会出现在第 1 名, 扩大候选范围也没有意义。这两点都要先用真实数据、真实训好的 checkpoint ([decompose/bart/outputs/bart_decomposer_musique_2wiki_repro/](#)) 跑一次验证, 具体看: top-N 候选之间的两两 BLEU (低=多样性高)、以及"best-of-N"随 N 变化的趋势 (N=1/3/5/10 时, N 个候选里最接近 GT 的那个分数/命中率有没有随 N 增大而明显变好)。验证通过 (候选确实多样、且扩大范围后确实更容易覆盖到接近正确的拆解) 才值得往下做方案 A 剩下的部分; 如果验证不通过, 方案 A 大概率要换成先改 BART 的解码策略 (比如换成 diverse beam search) 才有意义。这一步还没跑, 是接下来最先要做的事。

- BART 推理时把 `num_return_sequences` 从 1 改成 M (比如 3), 拿到 M 个候选拆解——这一步几乎不增加算力, 因为 BART 默认 `num_beams=10`, beam search 内部本来就在探索 10 条候选, 现在只是把第 2、3 名也捞出来, 不是重新生成。
- beam 的起点从"1 条空路径"变成"M 条种子路径", 每条路径除了记录已选证据/答案/累计分, 还额外带着"自己是用哪个候选拆解生成的" (自己的 `sub_questions` 列表)。
- 第 1 跳照常: M 条路径各自展开子问题 (不同拆解此刻文本可能不同) → 各自检索 → 一起打包批量过 gate 打分 → 全局剪枝到 B 条。这一步没有专门比较"哪个拆解更好", 纯粹按打分排序——拆解写得烂, 检索出来的证据接上去大概率也会显得不连贯, 异常分偏高, 自然被剪掉, 不需要额外逻辑。
- 剪枝后幸存的路径继续沿着自己所属的那份拆解往后走 (第 2 跳用这份拆解自己的第 2 个子问题), 不同路径可能来自不同拆解, 互不影响。
- **拆解跳数不一致的问题:** 如果拆解 A 是 2 跳、拆解 B 是 3 跳, 路径分数不能直接比"累计异常分之和" (跳数少的天然分低, 不公平), 改成比"每跳平均异常分"。但平均值本身还有一个更细的坑: 跳数少的路径打分次数少, 平均值方差大, 容易"运气好"出现虚低的平均分, 统计上天然偏向跳数少的路径——这个问题目前打算先不处理, 跑真实数据看偏差有多大再说, 同时加一条"单跳异常分超过阈值直接淘汰"的硬性保底规则, 防止"一步错得很明显, 但被其余跳的低分平均掉"这种情况。
- 最终答案: 跟方案一的 beam search 一样, 把不同拆解、不同长度、走到底的所有路径放在一起, 按"每跳平均异常分" (过滤掉被保底规则淘汰的) 选最优, 生成最终答案。

方案 B (更直接, 但成本高很多, 暂缓): 专门为"子问题本身好不好"训一个新的、独立于 evidence gate 的信号——在还没检索证据之前, 就用"prefix + 候选子问题"的隐状态判断这个子问题接不接得上前面的推理。这个信号更直接、更早, 但没有现成标签, 需要仿照现有 gold/counterfactual 协议, 人为构造一批"坏子问

题"（比如从别的题或别的跳错位插入一个子问题）训练数据，重新抽隐状态、重新训一个 PCA+LR，相当于给"拆解质量"单独造一个平行于现在 gate 的新模块，工作量明显大于方案 A。

顺序：先做方案 A（零训练成本，直接在方案一 beam search 基础上小改），如果发现"间接评判"效果 不够（比如异常分主要被证据牵着走，子问题本身的问题被稀释掉了、beam 剪枝救不回来明显烂的拆解），再考虑方案 B。

实现状态

设计已经讨论清楚，**代码还没写**——落地时会在

`error_propagation_probe/beam_search_retrieval.py` 基础上改（`BeamPath` 加一个自带的 `sub_questions` 字段，beam 初始化从 1 条改成 M 条，加一个 BART `num_return_sequences` 的小脚本产出候选拆解），不需要另起一个独立系统。

五、下一步

1. 验证 **BART beam** 候选的多样性和"扩大范围能不能覆盖到正确拆解"（见第四节方案 A 里的前置验证要求）——这是做"新思路 2 / 方案 A"之前必须先跑的一步，不需要等 beam search（第三节）先跑完，两者互不阻塞，可以并行推进。
`error_propagation_probe/check_decompose_diversity.py` 已经写好（复用 `decompose/bart/data_utils.py` 的数据加载/解析，读现成的 `bart_decomposer_musique_2wiki_repro` checkpoint，不训练不改动 `decompose` 代码），**还没有实际跑出结果**，是接下来要做的第一件事。
2. `beam_search_retrieval.py` 拿真实 GPU 跑一次，看跟现有 `gated_rule_a/lr_rerank` 比 EM/F1 有没有提升——这是验证"beam search 这个框架本身有没有用"的第一步。
3. 如果 1 验证通过、且 2 有正向结果，再实现"新思路 2 / 方案 A"剩下的部分（真正把拆解折进 beam），成本很低，可以很快跟进；如果 1 验证不通过，需要先考虑换 BART 解码策略（如 `diverse beam search`）。
4. `error_propagation_probe/` 里之前那三个"错误传导"分析脚本（`build_multi_error_traces.py` / `extract_full_hidden_states.py` / `score_and_analyze.py`）还没跑过真实数据，跟上面几条是独立的探索线，不互相阻塞。
5. `artifacts_pooled_v2` 转不转正，还是那个悬而未决的决策——第一节的数据已经给全了，等一个人决定。